```
import ast

def is_variable(x):
    """Returns True if the element is a variable (lowercase string)."""
    return isinstance(x, str) and x.islower()

def is_constant(x):
    """Returns True if the element is a constant (capitalized string)."""
    return isinstance(x, str) and not x.islower()

def occurs_in(var, expr):
    """Checks if a variable occurs inside an expression (Occur Check)."""
    if var == expr:
        return True
    if isinstance(expr, list):
        return any(occurs_in(var, sub) for sub in expr)
    return False

def apply_subst(subst, expr):
    """Applies a substitution set to an expression recursively."""
    if not subst or subst == "FAILURE":
        return expr
    if isinstance(expr, str):
        return subst.get(expr, expr)
    if isinstance(expr, list):
        return [apply_subst(subst, sub) for sub in expr]
    return expr

def unify(psi1, psi2):
    """
    Implements the First-Order Logic Unification Algorithm from the slides.
    """
    # Step 1: If psi1 or psi2 is a variable or constant
    if isinstance(psi1, str) or isinstance(psi2, str):
        # a) If they are identical, return NIL (empty dict)
        if psi1 == psi2:
            return {}

        # b) Else if psi1 is a variable
        elif is_variable(psi1):
            if occurs_in(psi1, psi2):
                return "FAILURE"
            else:
                return {psi1: psi2}

        # c) Else if psi2 is a variable
        elif is_variable(psi2):
            if occurs_in(psi2, psi1):
                return "FAILURE"
            else:
                return {psi2: psi1}

        # d) Else return FAILURE (both are distinct constants)
        else:
            return "FAILURE"

    # Handling list-based compound expressions/predicates
    if isinstance(psi1, list) and isinstance(psi2, list):
        # Step 2: If the initial Predicate symbol is not the same, return FAILURE
        if psi1[0] != psi2[0]:
            return "FAILURE"

        # Step 3: If they have a different number of arguments, return FAILURE
        if len(psi1) != len(psi2):
            return "FAILURE"

        # Step 4: Set Substitution set (SUBST) to NIL ({})
        subst = {}

        # Step 5: For i=1 to the number of elements
        for i in range(1, len(psi1)):
            # Apply current substitutions to arguments before unifying next elements
            arg1 = apply_subst(subst, psi1[i])
            arg2 = apply_subst(subst, psi2[i])
```

```
            # a) Call unify function recursively
            s = unify(arg1, arg2)

            # b) If S = failure then return FAILURE
            if s == "FAILURE":
                return "FAILURE"

            # c) If S != NIL then update and compose SUBST
            if s != {}:
                # Apply new substitutions to all existing values in the substitution set
                for k in subst:
                    subst[k] = apply_subst(s, subst[k])
                # Append/merge the new substitution
                subst.update(s)

        # Step 6: Return SUBST
        return subst

    return "FAILURE"

def parse_input(input_str):
    """Safely converts user string into a Python list format."""
    try:
        return ast.literal_eval(input_str.strip())
    except Exception:
        print("❌ Invalid format! Please input expressions exactly like a Python list.")
        return None

# ==========================================================================
# CONTINUOUS INTERACTIVE TESTER
# ==========================================================================
print("============================================================")
print("   🤖 UNIFICATION ALGORITHM INTERACTIVE LAB TESTER 🤖        ")
print("============================================================")
print("Rules for writing inputs:")
print("• Use standard list format: ['Predicate', 'Arg1', 'Arg2']")
print("• Lowercase are variables: 'x', 'y'")
print("• Capitalized are constants: 'Apple', 'Riya', 'John'")
print("============================================================\n")

while True:
    print("\n--- Enter Expressions (or type 'exit' to quit) ---")
    str1 = input("Expression 1: ")
    if str1.lower() == 'exit':
        print("Exiting program. Goodbye!")
        break

    str2 = input("Expression 2: ")
    if str2.lower() == 'exit':
        print("Exiting program. Goodbye!")
        break

    expr1 = parse_input(str1)
    expr2 = parse_input(str2)

    if expr1 is not None and expr2 is not None:
        print("\nEvaluating Unification...")
        result = unify(expr1, expr2)

        print("-------------------- RESULT --------------------")
        if result == "FAILURE":
            print("❌ STATUS: UNIFICATION FAILED")
        else:
            print("✅ STATUS: UNIFICATION SUCCESSFUL")
            print(f"📌 SUBST Set: {result if result != {} else 'NIL (Expressions are already identical)'}")
        print("------------------------------------------------")
```

```
============================================================
   🤖 UNIFICATION ALGORITHM INTERACTIVE LAB TESTER 🤖
============================================================
Rules for writing inputs:
• Use standard list format: ['Predicate', 'Arg1', 'Arg2']
• Lowercase are variables: 'x', 'y'
• Capitalized are constants: 'Apple', 'Riya', 'John'
============================================================
```

```
--- Enter Expressions (or type 'exit' to quit) ---
Expression 1: ['Eats','Riya','Apple']
Expression 2: ['Eats','John','Apple']

Evaluating Unification...
-------------------- RESULT --------------------
❌ STATUS: UNIFICATION FAILED
------------------------------------------------

--- Enter Expressions (or type 'exit' to quit) ---
Expression 1: ['Eats','Riya','Apple']
Expression 2: ['Eats','Riya','Apple']

Evaluating Unification...
-------------------- RESULT --------------------
✅ STATUS: UNIFICATION SUCCESSFUL
📌 SUBST Set: NIL (Expressions are already identical)
------------------------------------------------

--- Enter Expressions (or type 'exit' to quit) ---
Expression 1: exit
Exiting program. Goodbye!
```